

Otimização de balanceadores de carga de sistemas de distribuição de conteúdo por meio de monitoramento sistemático de memória

Pedro Faria Fernandes

Universidade do Estado de Santa Catarina

30 de novembro de 2025

Sumário I

Introdução

Fundamentação Teórica

Trabalhos Relacionados

Proposta

Implementação

Experimentos e Resultados

Conclusão

Referências

Introdução

Contexto e Motivação

Evolução do Consumo de Vídeo

- ▶ Da **televisão** tradicional para a Internet;
- ▶ Consolidação do **vídeo sob demanda** como padrão;
- ▶ Crescimento **exponencial** do tráfego de vídeo na rede;

Distribuição de Conteúdo

- ▶ Replicação de conteúdo em múltiplos servidores;
- ▶ Redes de Distribuição de Conteúdo (**CDNs**) como solução central;
- ▶ Necessidade de **otimizar recursos** para manter a Qualidade de Experiência (QoE);

O Problema

- ▶ Sistemas de distribuição de conteúdo $\approx$ múltiplos servidores e um **balanceador de carga** central;
- ▶ Dispositivos de armazenamento persistente (discos) frequentemente são **gargalos** em cenários de *streaming*;
- ▶ Uso intensivo de **memória principal** como *cache* alivia o disco, mas torna a memória um recurso crítico;
- ▶ Muitas estratégias clássicas de balanceamento de carga ignoram o estado real de memória dos servidores;
- ▶ **Lacuna**: ausência de algoritmos que utilizem **exclusivamente** métricas de memória (RAM e leitura de disco) como parâmetro principal de decisão.

Objetivo

Objetivo do trabalho

- ▶ Avaliar uma estratégia de otimização da escolha de servidores em balanceadores de carga para sistemas de distribuição de conteúdo, baseada **exclusivamente** no uso de memória;
- ▶ Implementar um algoritmo probabilístico de balanceamento de carga, apoiado em uma arquitetura de simulação completa;
- ▶ Realizar análise comparativa com algoritmos clássicos (*Round-Robin* e Seleção Aleatória) sob diferentes cenários de carga e heterogeneidade.

Fundamentação Teórica

Tópicos Centrais

- ▶ Balanceadores de carga em diferentes camadas (rede e aplicação) e balanceamento dinâmico [8, 14];
- ▶ Padrão **MPEG-DASH** para *streaming* adaptativo sobre HTTP [2, 16, 17];
- ▶ Arquitetura de **CDNs** e servidores de distribuição de conteúdo [15];
- ▶ Monitoramento de memória via APIs de sistemas operacionais (linux) [19, 18];
- ▶ Interpretação de dados de carga desatualizados [5, 6, 12];
- ▶ Virtualização de redes baseada em *containers* para simulação controlada [4].

Trabalhos Relacionados

Trabalhos Relacionados (Parte 1)

Título	Proposta	Limitações
NGINX [10, 13]	Algoritmos clássicos de balanceamento HTTP (<i>Round-Robin</i> , <i>Least Connections</i> , <i>IP hash</i>).	Não usa métricas de consumo de memória e leitura de disco para decisões de roteamento.
HAProxy [7]	Balanceador TCP/HTTP de alta eficiência e baixo consumo de recursos.	Não oferece balanceamento nativo guiado por uso de memória dos servidores.
Docker Swarm (memória) [3]	Acrescenta consumo de memória como fator no balanceamento interno do Swarm.	Restrito ao Docker; não trata CDNs nem métricas diretas de QoE em vídeo.
Algoritmo dinâmico em cloud [1]	Usa múltiplas métricas (CPU, memória, largura de banda, etc.) para lidar com <i>flash crowds</i> .	Focado em aplicações cloud genéricas; não é específico para servidores de conteúdo nem para gargalos de memória.

Trabalhos Relacionados (Parte 2)

Título	Proposta	Limitações
Revisão de algoritmos dinâmicos [11]	Compara 15 algoritmos de balanceamento dinâmico para servidores web.	Poucos usam memória/disco, e nenhum trata esses recursos como parâmetro principal exclusivo.
Sistema de replicação de conteúdo dinâmico [9]	Replica vídeos populares e balanceia carga com base na largura de banda de saída.	Considera predominantemente banda de rede; ignora gargalos de memória e disco.
Servidores de vídeo distrib. [20]	Duas fases: alocação inicial de réplicas e <i>load shifting</i> por streams ativos .	Mede carga apenas por número de streams; ignora uso real de memória taxa de leitura de disco.

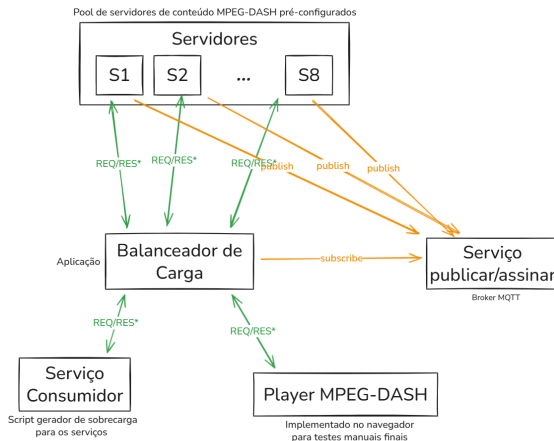
Proposta

Requisitos Tecnológicos

Componentes Principais

- ▶ **Servidores de Conteúdo:** 8 servidores MPEG-DASH, com capacidades variadas de memória e disco (de acordo com os cenários de teste);
- ▶ **Balanceador de Carga:** Implementado em *Rust*, responsável por aplicar os algoritmos de seleção de servidor;
- ▶ **Cliente Gerador de Carga:** Aplicação em *Python* que simula múltiplos usuários MPEG-DASH concorrentes;
- ▶ **Servidor MQTT:** *Broker* NanoMQ utilizado para transporte assíncrono de métricas entre servidores e balanceador.

Arquitetura Geral do Sistema



* Requisição e Resposta de conteúdo em áudio/vídeo

Arquitetura geral do sistema experimental (autor, 2025).

Objetivo do Algoritmo

- ▶ Definir o método de escolha de um servidor para cada nova requisição de vídeo;
- ▶ Algoritmo de natureza **probabilística**, baseado no **Basic Load Interpretation** [5];
- ▶ Probabilidade de escolha de cada servidor ajustada dinamicamente de acordo com:
 - ▶ Consumo de **memória RAM**;
 - ▶ Taxa de leitura de **disco**;
 - ▶ **Idade** das informações de carga.
- ▶ Servidores publicam periodicamente suas métricas de carga para o balanceador.

Cálculo da Probabilidade

A probabilidade (P_i) de uma requisição ser enviada para um servidor i é baseada no **Basic Load Interpretation**:

$$P_i = \frac{\frac{L_{tot} + arrive_T}{n} - L_i}{arrive_T} \quad (1)$$

P_i : Probabilidade de escolha do servidor i ;

n : Número total de servidores disponíveis;

L_i : Carga individual reportada pelo servidor i ;

L_{tot} : Somatório de todas as cargas reportadas ($\sum L_i$);

$arrive_T$: Número esperado de novas requisições durante o intervalo médio T das informações de carga.

Cálculo de $arrive_T$

O parâmetro $arrive_T$ representa o número esperado de requisições durante o intervalo de desatualização:

$$arrive_T = \left(\frac{R_{total}}{T_{elapsed}} \right) \cdot T_{stale} \cdot \left(\frac{L_{tot}}{R_{active}} \right) \cdot C \quad (2)$$

R_{total} : Total de requisições recebidas pelo balanceador (em segundos);

$T_{elapsed}$: Tempo total de execução do balanceador (em segundos);

T_{stale} : Tempo médio de desatualização das métricas (em segundos);

R_{active} : Requisições ativas;

C : Constante de ajuste (definida heurísticamente).

Adaptação do Parâmetro de Carga (L_i)

No contexto deste trabalho, a **carga** (L_i) é medida a partir do uso de memória:

$$L_i = CM \cdot LM_i + CD \cdot LD_i \quad (3)$$

LM_i : Consumo de **consumo de memória** normalizada do servidor i (0–1);

LD_i : Carga de **leitura disco** normalizada do servidor i (0–1);

CM, CD : Coeficientes dinâmicos que refletem a importância relativa de memória e disco.

Cálculo dos Coeficientes Dinâmicos (CM e CD)

Os coeficientes aumentam o peso do recurso mais pressionado no sistema como um todo.

- Taxa média de uso de memória:

$$TM = \frac{1}{n} \sum_{i=1}^n LM_i$$

- Taxa média de uso de disco:

$$TD = \frac{1}{n} \sum_{i=1}^n LD_i$$

$$CM = \frac{TM}{TM + TD} \quad (4)$$

$$CD = \frac{TD}{TM + TD} \quad (5)$$

Pseudocódigo — Atualização da Distribuição

Algorithm Atualização da distribuição de probabilidade

Gatilho: Nova informação de carga via MQTT.

Require: Métricas LM_i, LD_i, T_i, AR_i para cada servidor S_i

Ensure: Probabilidades P_i atualizadas

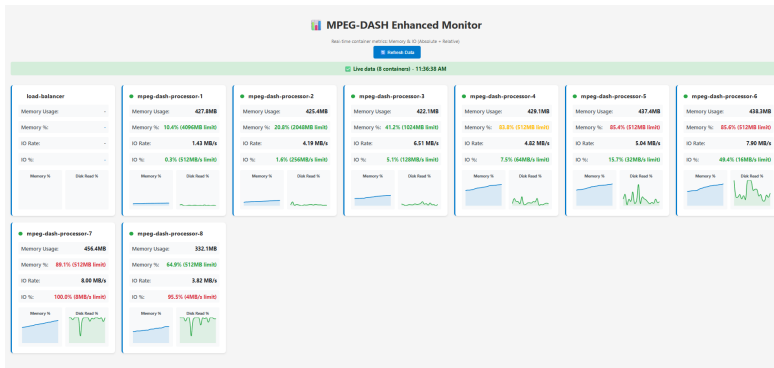
- 1: **procedure** ATUALIZARDISTRIBUICAO(LM_i, LD_i, T_i, AR_i)
 - 2: Atualizar métricas do servidor S_i
 - 3: $TM \leftarrow \frac{1}{n} \sum_{i=1}^n LM_i, \quad TD \leftarrow \frac{1}{n} \sum_{i=1}^n LD_i$
 - 4: $T \leftarrow \frac{1}{n} \sum_{i=1}^n (T_{atual} - T_i)$
 - 5: Calcular CM e CD (Eq. 4, 5)
 - 6: Calcular L_i para cada servidor (Eq. 3)
 - 7: $L_{tot} \leftarrow \sum_{i=1}^n L_i$
 - 8: Calcular $arrive_T$ (Eq. 2)
 - 9: Calcular P_i para cada servidor (Eq. 1)
 - 10: **if** $n = 0$ **ou** $arrive_T \leq 0$ **then**
 - 11: $P_i \leftarrow 1/n$ para todos os servidores
-

Implementação

Ambiente Experimental

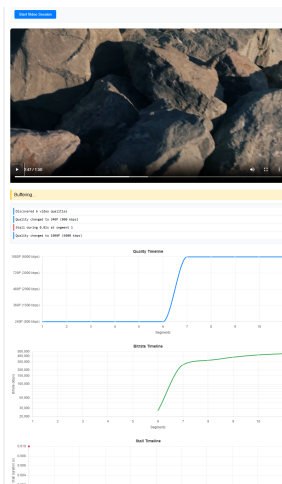
- ▶ Ambiente em **Windows + WSL2**, todos os componentes em contêineres **Docker**;
- ▶ Uso de **cgroups v2** para monitorar e limitar recursos (memória, leitura de disco);
- ▶ 8 servidores MPEG-DASH (*ASP.NET Core*) com conteúdo exclusivo via *bind mounts*;
- ▶ Balanceador de carga em *Rust*, gerador de carga em *Python*;
- ▶ Métricas publicadas via **MQTT** (NanoMQ) a cada 6s ou 30s;
- ▶ Conteúdo MPEG-DASH com 6 resoluções (144p a 1080p) gerado via *ffmpeg*.

Painel de Monitoramento em Tempo Real



Painel de monitoramento em tempo real com métricas de memória, leitura de disco e requisições ativas.

Cliente MPEG-DASH para Validação



Cliente MPEG-DASH (dash.js) com gráficos em tempo real de qualidade, *bitrate* e travamentos (*stalls*).

Experimentos e Resultados

Cenários de Teste

Cenário	Usuários	Distribuição	Configuração dos Servidores
1	100	Heterogênea	S1: 4096MB / 1024MB/s; S2: 2048MB / 512MB/s; S3: 1024MB / 256MB/s; S4: 512MB / 128MB/s; S5: 256MB / 64MB/s; S6: 128MB / 32MB/s; S7: 64MB / 16MB/s; S8: 32MB / 8MB/s
2	100	Homogênea	Todos os servidores: 1024MB RAM e 256MB/s de leitura
3	1000	Heterogênea	Mesma configuração do Cenário 1
4	1000	Homogênea	Mesma configuração do Cenário 2

Estratégias de Balanceamento

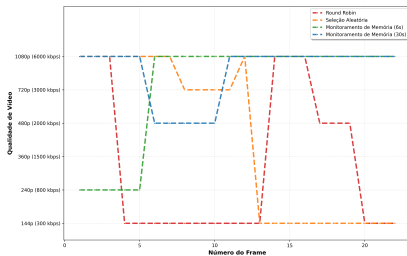
- ▶ **Round-Robin**: distribuição cíclica uniforme;
- ▶ **Seleção Aleatória**: servidor escolhido de forma uniforme e aleatória a cada requisição;
- ▶ **Monitoramento de Memória (6s)**: algoritmo proposto com atualização frequente de métricas;
- ▶ **Monitoramento de Memória (30s)**: algoritmo proposto com dados moderadamente desatualizados.

Métricas de Avaliação de QoE

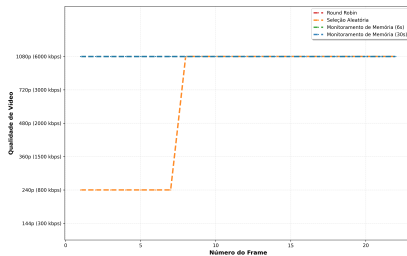
- ▶ **Latência Média** de resposta por requisição HTTP;
- ▶ **Qualidade** de vídeo ao longo da sessão (resoluções escolhidas pelo cliente);
- ▶ **Bitrate** médio por segmento;
- ▶ **Travamentos** (*stalls*): quantidade, duração total e maior duração.

Qualidade de Vídeo por Cenário

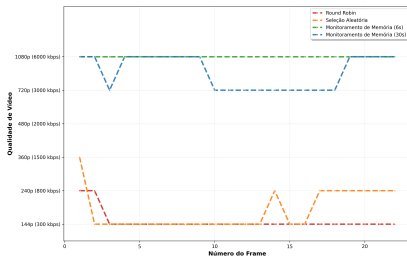
Qualidade de Vídeo por Frame - Cenário 1



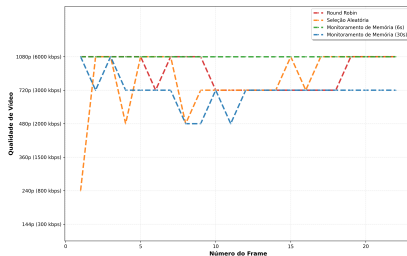
Qualidade de Vídeo por Frame - Cenário 2



Qualidade de Vídeo por Frame - Cenário 3

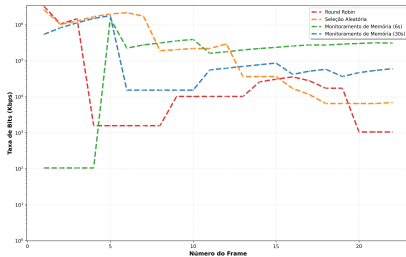


Qualidade de Vídeo por Frame - Cenário 4

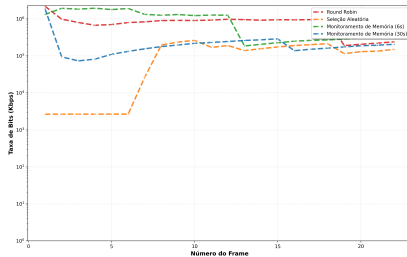


Bitrate por Cenário

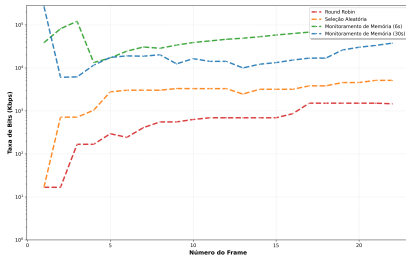
Taxa de Bits por Frame - Cenário 1



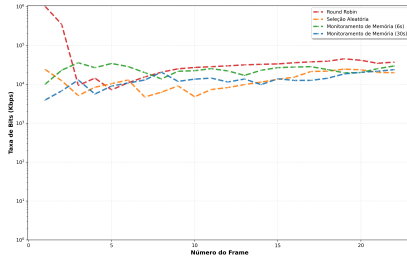
Taxa de Bits por Frame - Cenário 2



Taxa de Bits por Frame - Cenário 3

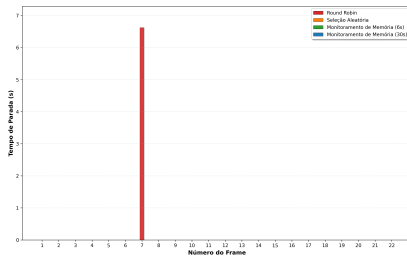


Taxa de Bits por Frame - Cenário 4

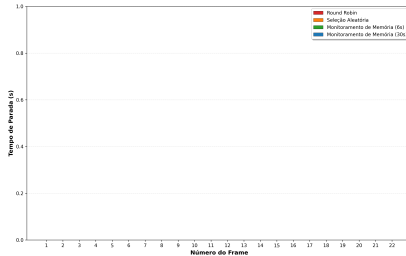


Stalls por Cenário

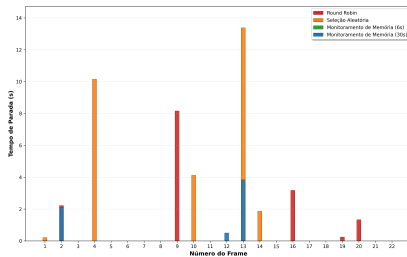
Duração de Parada por Frame - Cenário 1



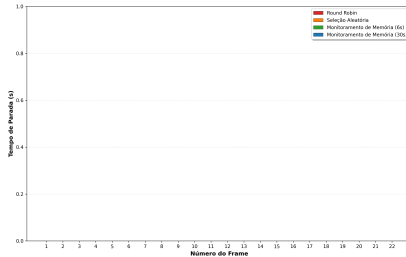
Duração de Parada por Frame - Cenário 2



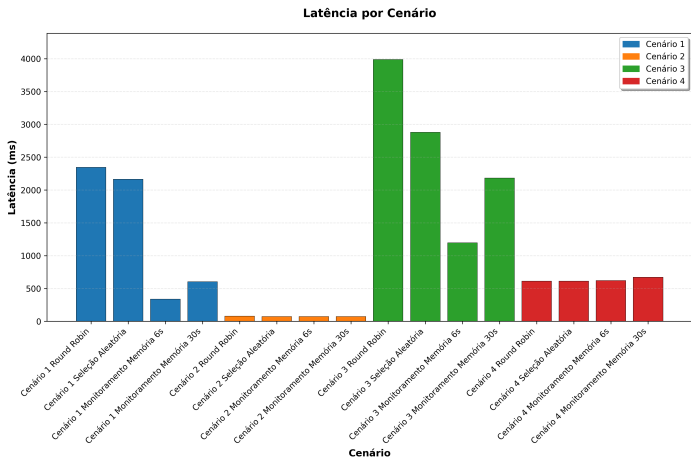
Duração de Parada por Frame - Cenário 3



Duração de Parada por Frame - Cenário 4



Latência Média por Cenário



Latência média de resposta por cenário e estratégia de balanceamento.

Conclusão

Fatores Determinantes do Desempenho

- ▶ **Heterogeneidade dos servidores:**

- ▶ Quanto maior a diferença de capacidade entre nós, maior o ganho do balanceamento inteligente;

- ▶ **Nível de concorrência:**

- ▶ Alta concorrência funciona como teste de estresse, expondo limitações de algoritmos simples;
- ▶ O algoritmo proposto mantém desempenho superior mesmo em cenários extremos (1000 usuários).

Contribuições do Trabalho

- ▶ Proposição de um **algoritmo de balanceamento** baseado em monitoramento de memória e leitura de disco;
- ▶ Adaptação do conceito de **Load Interpretation** para métricas de recursos de baixo nível;
- ▶ Implementação completa e aberta com *C#*, *Rust*, *Python*, Docker e MQTT;
- ▶ Infraestrutura experimental reutilizável para avaliação de algoritmos de balanceamento em *streaming* MPEG-DASH;
- ▶ Demonstração de que heterogeneidade é fator crítico para o valor de estratégias inteligentes de balanceamento.

Limitações e Trabalhos Futuros

- ▶ Experimentos conduzidos em ambiente virtualizado único, sem latência geográfica real;
- ▶ Uso de um único algoritmo ABR (`dash.js`) e conjunto limitado de cenários de carga;
- ▶ Métricas focadas em memória e disco, sem considerar rede e CPU como co-fatores;
- ▶ **Trabalhos futuros:**
 - ▶ Validação em ambientes distribuídos reais e CDNs de produção;
 - ▶ Extensão para múltiplas métricas (CPU, rede, latência geográfica);
 - ▶ Integração com SDN e *Content Steering* entre múltiplas CDNs;
 - ▶ Aplicação em cenários de *edge computing* e 5G.

Reprodutibilidade e Considerações Finais

- ▶ Todo o código, scripts e dados estão disponíveis em:
 - ▶ <https://github.com/peedrofernandes/memory-oriented-load-balancer.git>
- ▶ A infraestrutura experimental permite reproduzir todos os 16 cenários de teste;
- ▶ Resultados demonstram que **monitorar e interpretar adequadamente o uso de recursos** é essencial para QoE em CDNs heterogêneas;
- ▶ Estratégias de balanceamento conscientes de memória representam um caminho promissor para a próxima geração de sistemas de distribuição de conteúdo.

Referências

Referências I

- [1] Adekunbi Adewojo and Julian Bass. A novel weight-assignment load balancing algorithm for cloud applications. *SN Computer Science*, 4, 03 2023.
- [2] Ali Begen, Tankut Akgul, and Mark Baugher. Watching video over the web: Part 1: Streaming protocols. *IEEE Internet Computing*, 15(2), 2011.
- [3] Mochamad Rexa Mei Bella, Mahendra Data, and Widhi Yahya. Web server load balancing based on memory utilization using docker swarm. In *2018 International Conference on Sustainable Information Engineering and Technology (SIET)*, 2018.
- [4] N.M. Mosharaf Kabir Chowdhury and Raouf Boutaba. A survey of network virtualization. *Computer Networks*, 54(5), 2010.
- [5] M. Dahlin. Interpreting stale load information. *IEEE Transactions on Parallel and Distributed Systems*, 11(10), 2000.

Referências II

- [6] Derek L. Eager, Edward D. Lazowska, and John Zahorjan. Adaptive load sharing in homogeneous distributed systems. *IEEE Transactions on Software Engineering*, SE-12(5), 1986.
- [7] HAProxy. Haproxy - the reliable, high performance tcp/http load balancer, 2025. Acessado em: 27 de março de 2025.
- [8] Kimberly Jane. Scalability issues in database systems: Strategies for scaling databases horizontally and vertically. 03 2025.
- [9] Junyeop Kim and Youjip Won. Dynamic load balancing for efficient video streaming service. In *2015 International Conference on Information Networking (ICOIN)*, pages 216–221, 2015.
- [10] Chen Ma and Yuhong Chi. Evaluation test and improvement of load balancing algorithms of nginx. *IEEE Access*, 10:14311–14324, 2022.
- [11] Ibrahim Mahmood, Siddeeq Ameen, Hajar Yasin, Naaman Omar, Shakir Kak, Zryan Najat, Azar Salih, Nareen O. M.Salim, and Dindar Ahmed. Web server performance improvement using dynamic load balancing techniques: A review. *Asian Journal of Research in Computer Science*, 06 2021.

Referências III

- [12] R. Mirchandaney, D. Towsley, and J.A. Stankovic. Analysis of the effects of delays on load sharing. *IEEE Transactions on Computers*, 38(11), 1989.
- [13] NGINX. Http load balancer - nginx documentation, 2025. Accessed: 2025-03-27.
- [14] A. Pasumpon Pandian, Xavier Fernando, and Wang Haoxiang, editors. *Computer Networks, Big Data and IoT: Proceedings of ICCBI 2021*, volume 117 of *Lecture Notes on Data Engineering and Communications Technologies*. Springer Nature Singapore, 2022.
- [15] Sam Preston. Finding the best servers to answer queries—edge dns and anycast, March 2021. Akamai Blog.
- [16] Iraj Sodagar. The mpeg-dash standard for multimedia streaming over the internet. *IEEE MultiMedia*, 18(4), 2011.
- [17] Thomas Stockhammer. Dynamic adaptive streaming over http: Standards and design principles. 02 2011.

Referências IV

- [18] The Linux Kernel Developers. I/o statistics fields for block devices, January 2011. Descreve os campos fornecidos pelo arquivo `/proc/diskstats`.
- [19] The Linux Kernel Developers. The `/proc` filesystem, March 2020. Documentação do kernel sobre a estrutura e os arquivos do sistema de arquivos `/proc`, incluindo `/proc/meminfo`.
- [20] Shiao-Li Tsao, Meng Chang Chen, Ming-Tat Ko, Jan-Ming Ho, and Yueh-Min Huang. Data allocation and dynamic load balancing for distributed video storage server. *Journal of Visual Communication and Image Representation*, 10(2):197–218, 1999.